

Blog Platform with Comments

Full Stack Project Documentation

Project Objective

The objective of this project is to develop a full-stack blogging platform where users can create blog posts and interact through comments.

Users can:

- Register and Login
- Create Blog Posts
- Edit Blog Posts
- Delete Blog Posts
- Add Comments
- Interact with Other Users

Technologies Used

Frontend:

- React.js
- HTML
- CSS
- Bootstrap

Backend:

- Node.js
- Express.js

Database:

- MongoDB

Authentication:

- JWT Authentication

Key Features

- User Registration and Login
- JWT Authentication
- Create Blog Posts
- Edit Blog Posts
- Delete Blog Posts
- Comment Section for User Interaction
- RESTful APIs
- MongoDB Database Integration
- Responsive User Interface

Project Folder Structure

```
blog-platform/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Navbar.js
│   │   │   ├── PostForm.js
│   │   │   ├── CommentSection.js
│   │   │   └── PostList.js
│   │   ├── pages/
│   │   │   ├── Login.js
│   │   │   ├── Register.js
│   │   │   ├── Home.js
│   │   │   └── Dashboard.js
│   │   ├── App.js
│   │   ├── api.js
│   │   └── index.js
│   └──
├── server/
│   ├── models/
│   │   ├── User.js
│   │   ├── Post.js
│   │   └── Comment.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── postRoutes.js
│   │   └── commentRoutes.js
│   ├── server.js
│   └── package.json
```

Frontend Setup

```
npx create-react-app client

cd client

npm install axios react-router-dom bootstrap
```

index.js

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

api.js

```
import axios from "axios";

export default axios.create({
  baseURL: "http://localhost:5000/api"
});
```

App.js

```
import React from "react";

import { BrowserRouter, Routes, Route } from "react-router-dom";

import Login from "./pages/Login";
import Register from "./pages/Register";
import Home from "./pages/Home";
import Dashboard from "./pages/Dashboard";

function App() {

  return (

    <BrowserRouter>

      <Routes>

        <Route path="/" element={<Home />} />

        <Route path="/login" element={<Login />} />

        <Route path="/register" element={<Register />} />

        <Route path="/dashboard" element={<Dashboard />} />

      </Routes>

    </BrowserRouter>

  );
}

export default App;
```

Login.js

```
import React, { useState } from "react";
```

```

import api from "../api";

function Login() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const loginUser = async () => {

    const res = await api.post("/auth/login", {
      email,
      password
    });

    localStorage.setItem("token", res.data.token);

    alert("Login Successful");
  };

  return (

    <div>

      <h2>Login</h2>

      <input
        type="email"
        placeholder="Enter Email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        placeholder="Enter Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={loginUser}>
        Login
      </button>

    </div>

  );
}

export default Login;

```

Register.js

```

import React, { useState } from "react";

import api from "../api";

function Register() {

  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const registerUser = async () => {

    await api.post("/auth/register", {
      name,
      email,

```

```

        password
    });

    alert("Registration Successful");
};

return (
    <div>

        <h2>Register</h2>

        <input
            type="text"
            placeholder="Enter Name"
            onChange={(e) => setName(e.target.value)}
        />

        <input
            type="email"
            placeholder="Enter Email"
            onChange={(e) => setEmail(e.target.value)}
        />

        <input
            type="password"
            placeholder="Enter Password"
            onChange={(e) => setPassword(e.target.value)}
        />

        <button onClick={registerUser}>
            Register
        </button>

    </div>

);
}

export default Register;

```

Dashboard.js

```

import React, { useEffect, useState } from "react";

import api from "../api";

function Dashboard() {

    const [posts, setPosts] = useState([]);

    const [title, setTitle] = useState("");

    const [content, setContent] = useState("");

    useEffect(() => {
        fetchPosts();
    }, []);

    const fetchPosts = async () => {

        const res = await api.get("/posts");

        setPosts(res.data);
    };
}

```

```

const createPost = async () => {

  await api.post("/posts", {
    title,
    content
  });

  fetchPosts();
};

return (

  <div>

    <h2>Create Blog Post</h2>

    <input
      type="text"
      placeholder="Post Title"
      onChange={(e) => setTitle(e.target.value)}
    />

    <textarea
      placeholder="Post Content"
      onChange={(e) => setContent(e.target.value)}
    />

    <button onClick={createPost}>
      Create Post
    </button>

    {
      posts.map((post) => (

        <div key={post._id}>

          <h3>{post.title}</h3>

          <p>{post.content}</p>

        </div>

      ))
    }

  </div>

);
}

export default Dashboard;

```

CommentSection.js

```

import React, { useState } from "react";

import api from "../api";

function CommentSection({ postId }) {

  const [comment, setComment] = useState("");

  const addComment = async () => {

    await api.post("/comments", {

```

```

        postId,
        comment
    });

    alert("Comment Added Successfully");
};

return (
    <div>

        <input
            type="text"
            placeholder="Write Comment"
            onChange={(e) => setComment(e.target.value)}
        />

        <button onClick={addComment}>
            Add Comment
        </button>

    </div>

);
}

export default CommentSection;

```

Backend Setup

```

npm init -y

npm install express mongoose cors dotenv bcryptjs jsonwebtoken nodemon

```

server.js

```

const express = require("express");

const mongoose = require("mongoose");

const cors = require("cors");

require("dotenv").config();

const authRoutes = require("./routes/authRoutes");

const postRoutes = require("./routes/postRoutes");

const commentRoutes = require("./routes/commentRoutes");

const app = express();

app.use(cors());

app.use(express.json());

mongoose.connect(process.env.MONGO_URI)

    .then(() => console.log("MongoDB Connected"))

    .catch((err) => console.log(err));

app.use("/api/auth", authRoutes);

```

```
app.use("/api/posts", postRoutes);

app.use("/api/comments", commentRoutes);

const PORT = 5000;

app.listen(PORT, () => {

  console.log(`Server running on port ${PORT}`);

});
```

User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

  name: String,

  email: String,

  password: String

});

module.exports = mongoose.model("User", userSchema);
```

Post.js

```
const mongoose = require("mongoose");

const postSchema = new mongoose.Schema({

  title: String,

  content: String,

  createdAt: {
    type: Date,
    default: Date.now
  }

});

module.exports = mongoose.model("Post", postSchema);
```

Comment.js

```
const mongoose = require("mongoose");

const commentSchema = new mongoose.Schema({

  postId: String,

  comment: String,

  createdAt: {
    type: Date,
    default: Date.now
  }

});
```



```

    }

  });

  module.exports = mongoose.model("Comment", commentSchema);

```

authRoutes.js

```

const express = require("express");

const bcrypt = require("bcryptjs");

const jwt = require("jsonwebtoken");

const User = require("../models/User");

const router = express.Router();

router.post("/register", async (req, res) => {

  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = new User({
    name,
    email,
    password: hashedPassword
  });

  await user.save();

  res.json({
    message: "User Registered Successfully"
  });

});

router.post("/login", async (req, res) => {

  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
    return res.json({
      message: "User Not Found"
    });
  }

  const isMatch = await bcrypt.compare(password, user.password);

  if (!isMatch) {
    return res.json({
      message: "Invalid Password"
    });
  }

  const token = jwt.sign(
    { id: user._id },
    "secretkey"
  );

  res.json({ token });

});

```

```
module.exports = router;
```

postRoutes.js

```
const express = require("express");

const Post = require("../models/Post");

const router = express.Router();

router.get("/", async (req, res) => {

  const posts = await Post.find();

  res.json(posts);

});

router.post("/", async (req, res) => {

  const post = new Post(req.body);

  await post.save();

  res.json(post);

});

router.put("/:id", async (req, res) => {

  const updatedPost = await Post.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );

  res.json(updatedPost);

});

router.delete("/:id", async (req, res) => {

  await Post.findByIdAndDelete(req.params.id);

  res.json({
    message: "Post Deleted Successfully"
  });

});

module.exports = router;
```

commentRoutes.js

```
const express = require("express");

const Comment = require("../models/Comment");

const router = express.Router();

router.post("/", async (req, res) => {

  const comment = new Comment(req.body);
```

```
        await comment.save();

        res.json(comment);
    });

    router.get("/", async (req, res) => {

        const comments = await Comment.find();

        res.json(comments);
    });

    module.exports = router;
```

Environment Variables

```
MONGO_URI=your_mongodb_connection

JWT_SECRET=secretkey
```

Run Backend Server

```
cd server

npm install

npm run dev
```

Run Frontend Application

```
cd client

npm install

npm start
```

Conclusion

The Blog Platform with Comments project is a complete MERN stack application developed using React.js, Node.js, Express.js, and MongoDB.

This project helps students understand:

- Authentication Systems
- CRUD Operations
- REST API Development
- Database Integration

- User Interaction Systems
- Full Stack Development Concepts